

Day 3: Python AI Framework Ecosystem

1. 5-line beginner summary

Python is important in AI because most AI libraries, examples, models, and tools are built around Python. NumPy helps with fast numerical calculations, especially arrays and matrices. Pandas helps clean, analyze, and prepare tabular data like CSV, Excel, database extracts, and logs. Scikit-learn is used for traditional machine learning like classification, regression, clustering, and preprocessing. PyTorch, TensorFlow/Keras, and Hugging Face are mainly used for deep learning, LLMs, NLP, vision, and GenAI applications.

2. Big picture: Why Python is important in AI projects

Python is popular in AI because it is simple to write and has a very strong AI ecosystem.

In real AI projects, Python is used for:

Area	Python role
Data loading	Read CSV, JSON, database, APIs
Data cleaning	Remove missing values, fix formats, transform columns
Data analysis	Understand data patterns
Feature engineering	Create useful input columns for ML models
Model training	Train ML, deep learning, or GenAI models
Model evaluation	Check accuracy, precision, recall, F1, ROC-AUC
Model deployment	Serve model through API
MLOps	Track experiments, package models, monitor performance

Think of Python as the **main working language** for AI engineers and data scientists.

Simple example:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

data = pd.read_csv("customer_data.csv")

X = data[["age", "income", "visits"]]
y = data["will_buy"]

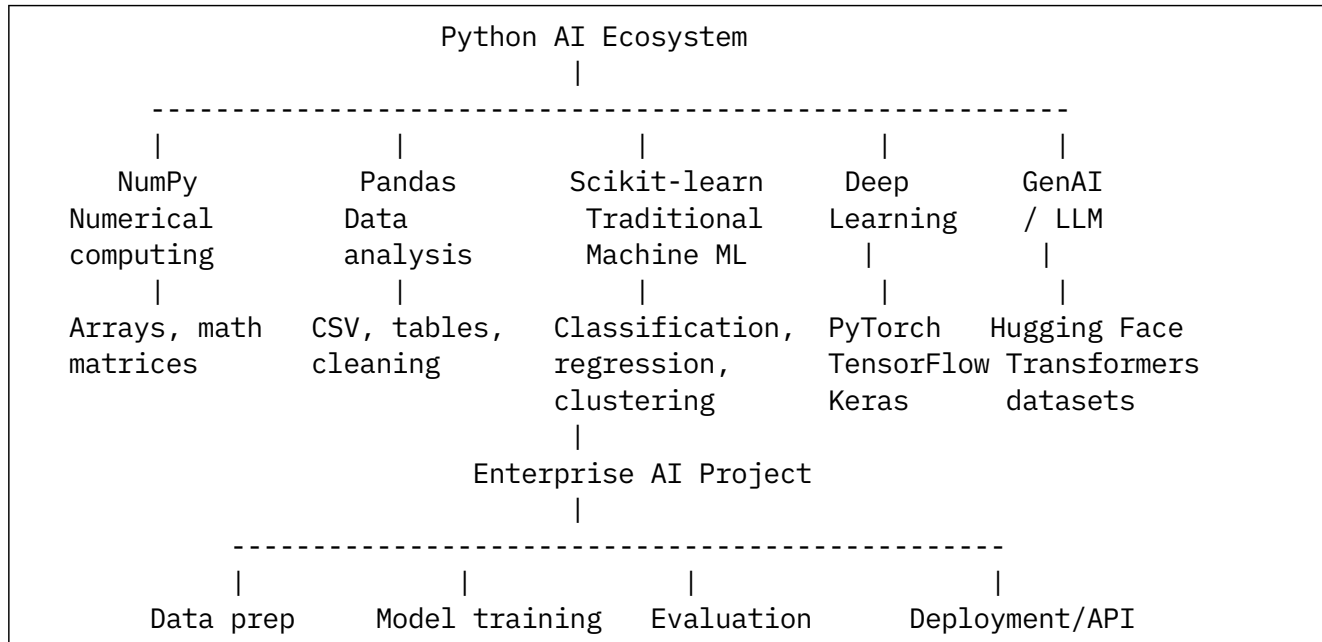
X_train, X_test, y_train, y_test = train_test_split(X, y)

model = RandomForestClassifier()
model.fit(X_train, y_train)
```

```
print(model.score(X_test, y_test))
```

This small script shows how Python connects data loading, ML training, and evaluation.

3. Python AI ecosystem ASCII diagram



4. NumPy for numerical operations

What is NumPy?

NumPy is a Python library used for fast mathematical operations.

It is mainly used for:

- Arrays
- Matrix operations
- Mathematical calculations
- Vector operations
- Numerical data processing

In normal Python, lists are flexible but slower for large numerical operations. NumPy arrays are faster and more suitable for AI/ML.

Easy example

Imagine you have customer spending data:

```
import numpy as np

spending = np.array([100, 200, 300, 400])

print(spending.mean())
```

```
print(spending.max())  
print(spending * 2)
```

Output idea:

```
250.0  
400  
[200 400 600 800]
```

Real AI usage

In AI, model input is usually converted into numbers.

Example:

```
Text: "good product"  
Converted to numbers: [0.23, 0.91, 0.11, 0.64]
```

These numbers are often handled using NumPy arrays or tensors.

Where NumPy fits

```
Raw data -> Numerical representation -> NumPy arrays -> ML/deep learning model
```

5. Pandas for data analysis

What is Pandas?

Pandas is used to work with tabular data.

Tabular data means rows and columns, like:

customer_id	age	salary	purchased
101	25	50000	yes
102	31	70000	no

Pandas is commonly used for:

- Reading CSV/Excel files
- Cleaning missing values
- Filtering rows
- Grouping data
- Creating new columns
- Basic exploratory data analysis, also called EDA

Easy example

```
import pandas as pd  
  
data = {  
    "name": ["Amit", "Priya", "John"],  
    "age": [25, 30, 35],  
    "salary": [50000, 70000, 90000]}
```

```
}  
  
df = pd.DataFrame(data)  
  
print(df.head())  
print(df["salary"].mean())
```

Example: cleaning missing values

```
df["salary"] = df["salary"].fillna(df["salary"].mean())
```

This means:

If salary is missing, replace it with average salary.

Real AI usage

Before training a model, Pandas is used to prepare data.

```
Raw CSV -> Pandas cleaning -> Features and labels -> ML model
```

Example:

```
X = df[["age", "salary"]]  
y = df["purchased"]
```

Here:

- X means input features
- y means target/output

6. Scikit-learn for traditional ML

What is Scikit-learn?

Scikit-learn is a popular Python library for traditional machine learning.

It is used for:

- Classification
- Regression
- Clustering
- Feature preprocessing
- Train/test split
- Cross-validation
- Model evaluation

When to use Scikit-learn

Use Scikit-learn when your data is mostly structured/tabular.

Examples:

Problem	ML type
Predict customer churn	Classification
Predict house price	Regression
Group customers by behavior	Clustering
Detect fraudulent transactions	Classification
Predict sales	Regression

Easy classification example

```
from sklearn.tree import DecisionTreeClassifier

X = [
    [25, 50000],
    [35, 80000],
    [45, 120000],
    [22, 30000]
]

y = [0, 1, 1, 0]

model = DecisionTreeClassifier()
model.fit(X, y)

prediction = model.predict([[30, 60000]])

print(prediction)
```

Meaning:

Input: age=30, salary=60000
Output: 0 or 1

Scikit-learn workflow

Data -> Clean -> Split -> Train model -> Evaluate -> Save model

Common Scikit-learn models

Task	Common models
Classification	Logistic Regression, Decision Tree, Random Forest, XGBoost-style models
Regression	Linear Regression, Random Forest Regressor
Clustering	KMeans, DBSCAN
Dimensionality reduction	PCA
Preprocessing	StandardScaler, OneHotEncoder

7. PyTorch basics

What is PyTorch?

PyTorch is a deep learning framework.

It is commonly used for:

- Neural networks
- Computer vision
- NLP
- LLM research
- Custom deep learning models
- GPU-based training

PyTorch is popular because it is flexible and easy to debug.

Important PyTorch concept: Tensor

A tensor is like a NumPy array, but it can run efficiently on GPU.

```
import torch

x = torch.tensor([1.0, 2.0, 3.0])
y = x * 2

print(y)
```

Output:

```
tensor([2., 4., 6.])
```

Simple neural network idea

```
import torch
import torch.nn as nn

model = nn.Linear(2, 1)

input_data = torch.tensor([[25.0, 50000.0]])
output = model(input_data)

print(output)
```

This means:

```
Input has 2 features.
Output has 1 prediction.
```

Where PyTorch is useful

Use PyTorch when:

- You need deep learning

- You need custom model architecture
 - You are working with images, text, audio, or embeddings
 - You want more flexibility
 - You are fine-tuning LLMs or transformer models
-

8. TensorFlow/Keras basics

What is TensorFlow?

TensorFlow is a deep learning framework created for building and deploying neural networks.

What is Keras?

Keras is a high-level API that makes TensorFlow easier to use.

Simple way to remember:

```
TensorFlow = powerful deep learning engine  
Keras = beginner-friendly interface on top of TensorFlow
```

Easy Keras example

```
import tensorflow as tf  
from tensorflow import keras  
  
model = keras.Sequential([  
    keras.layers.Dense(16, activation="relu"),  
    keras.layers.Dense(1, activation="sigmoid")  
)  
  
model.compile(  
    optimizer="adam",  
    loss="binary_crossentropy",  
    metrics=["accuracy"]  
)
```

This model can be used for binary classification, for example:

```
Will customer buy? yes/no  
Will user churn? yes/no  
Is transaction fraud? yes/no
```

When to use TensorFlow/Keras

Use TensorFlow/Keras when:

- You want to build deep learning models
- You prefer a structured high-level API
- You are working with production deep learning pipelines
- Your organization already uses TensorFlow
- You need deployment options like TensorFlow Serving or TensorFlow Lite

9. Hugging Face basics

What is Hugging Face?

Hugging Face is an ecosystem for modern AI models, especially transformer models.

It is widely used for:

- LLMs
- NLP
- Text classification
- Summarization
- Translation
- Question answering
- Embeddings
- Fine-tuning models
- Loading pretrained models

Easy example: sentiment analysis

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis")

result = classifier("This product is very good.")

print(result)
```

Possible output:

```
[{'label': 'POSITIVE', 'score': 0.99}]
```

Easy example: text generation

```
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

result = generator("AI is changing the world because", max_length=30)

print(result)
```

Hugging Face main components

Component	Purpose
Transformers	Use pretrained models
Datasets	Load public datasets
Tokenizers	Convert text into tokens

Component	Purpose
Hub	Store and download models/datasets
Accelerate	Train models efficiently
PEFT	Parameter-efficient fine-tuning

When to use Hugging Face

Use Hugging Face when:

- You need pretrained models
- You are working with NLP or LLMs
- You want text classification, summarization, Q&A, translation
- You want embeddings for RAG
- You want to fine-tune a transformer model
- You want to experiment quickly with open-source models

10. When to use Scikit-learn

Use Scikit-learn when your problem is traditional ML and your data is structured.

Good use cases

Customer churn prediction
Loan approval prediction
Sales forecasting
Fraud detection
Employee attrition prediction
Customer segmentation

Example decision

Data: customer age, salary, city, usage count, subscription type
Goal: predict churn yes/no
Best starting framework: Scikit-learn

Why?

Because this is tabular data and a classification problem.

Scikit-learn is good for:

- Fast experiments
- Baseline models
- Interpretable models
- Tabular data
- Small to medium ML projects
- Feature engineering pipelines

11. When to use PyTorch or TensorFlow

Use PyTorch or TensorFlow when Scikit-learn is not enough.

Good use cases

```
Image classification
Speech recognition
Large text models
Custom neural networks
Deep learning recommendation systems
Fine-tuning transformer models
Complex sequence modeling
```

Example decision

```
Data: product images
Goal: detect damaged items
Best framework: PyTorch or TensorFlow
```

Why?

Because image data usually needs deep learning.

PyTorch vs TensorFlow simple understanding

Framework	Simple understanding
PyTorch	Flexible, popular in research and custom modeling
TensorFlow/Keras	Structured, mature for production and deployment

Both can be used in enterprise AI projects.

12. When to use Hugging Face

Use Hugging Face when you want to use pretrained transformer models.

Good use cases

```
Summarize documents
Classify support tickets
Extract entities from text
Build chatbot backend
Generate embeddings for RAG
Fine-tune an LLM
Question answering on documents
```

Example decision

```
Data: customer support emails
Goal: classify emails into categories
Best framework: Hugging Face
```

Why?

Because pretrained language models already understand text patterns better than traditional ML in many NLP cases.

Hugging Face in GenAI projects

For a RAG application:

Documents -> Chunking -> Embeddings using Hugging Face model -> Vector DB -> LLM response

13. Simple comparison table

Framework	Main purpose	Best for	Example use case	Beginner memory trick
NumPy	Numerical computing	Arrays, math, matrix operations	Convert data into numerical arrays	Numbers
Pandas	Data analysis	Tables, CSV, cleaning	Clean customer dataset	Excel inside Python
Scikit-learn	Traditional ML	Classification, regression, clustering	Predict customer churn	Classic ML toolbox
PyTorch	Deep learning	Custom neural networks, LLM fine-tuning	Train image model	Flexible deep learning
TensorFlow/Keras	Deep learning	Neural networks, production pipelines	Build fraud detection neural net	Structured deep learning
Hugging Face	Pretrained AI models	NLP, LLMs, transformers	Summarize documents	Ready-made GenAI models

14. How these frameworks fit into enterprise AI projects

In enterprise AI projects, you usually do not use only one framework. You combine them.

Example: Customer churn prediction system

Pandas -> Load and clean customer data
NumPy -> Numerical operations
Scikit-learn -> Train churn prediction model
MLflow -> Track model experiments
FastAPI -> Expose model as REST API
Docker/K8s -> Deploy service
Monitoring -> Track accuracy and drift

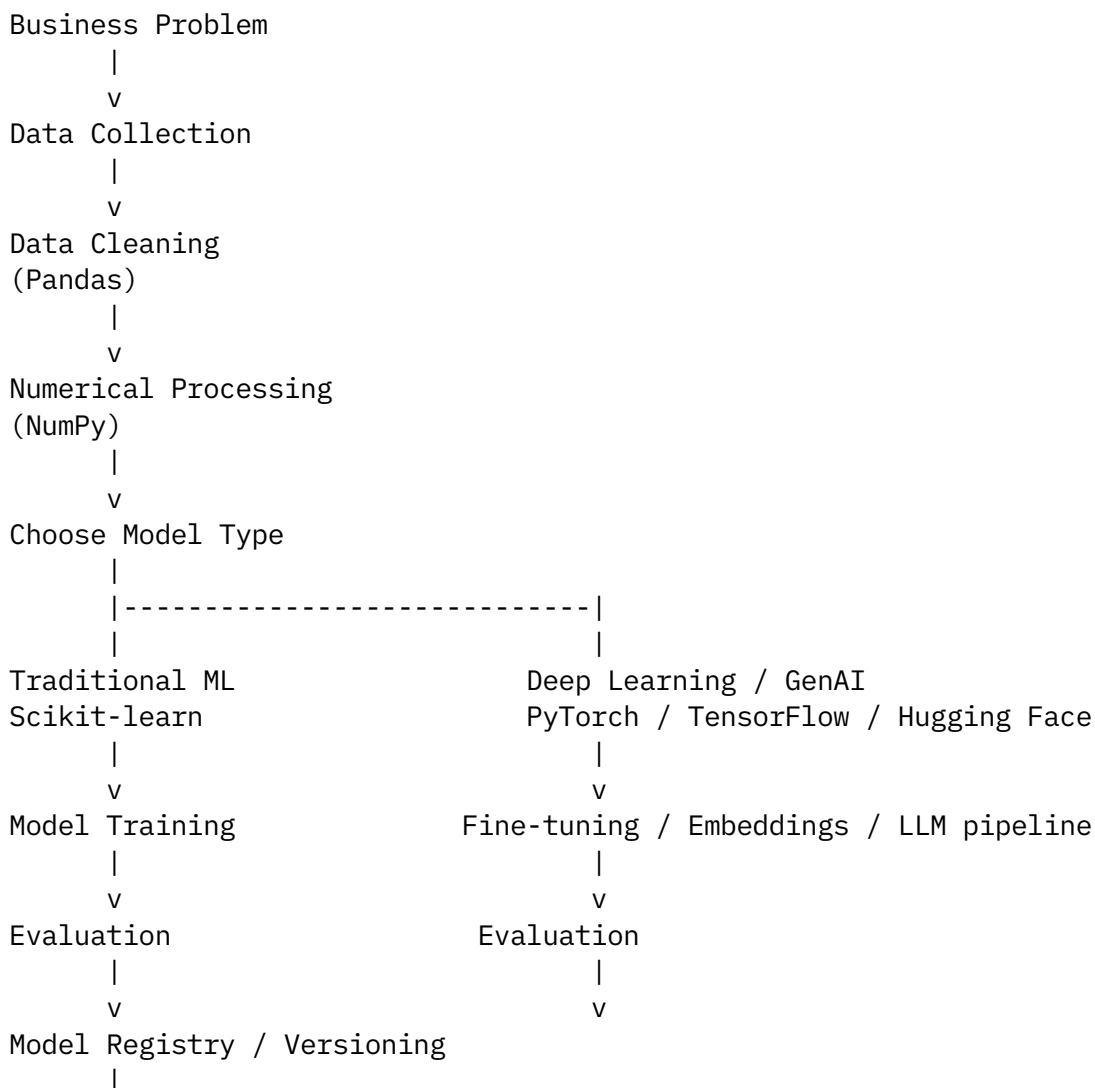
Example: Enterprise document Q&A using GenAI

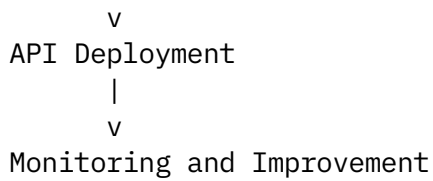
Pandas / Python scripts	-> Load metadata
Hugging Face tokenizer	-> Process text
Embedding model	-> Convert chunks into vectors
Vector database	-> Store embeddings
LLM	-> Generate final answer
FastAPI	-> Serve Q&A API
Monitoring	-> Track latency, cost, feedback

Example: Deep learning image project

Python	-> Main programming language
NumPy	-> Numerical processing
PyTorch	-> Train image model
Pandas	-> Track labels and metadata
MLflow	-> Track experiments
API	-> Serve prediction endpoint
Cloud	-> Scale training and deployment

15. Enterprise AI workflow using these frameworks





16. Pseudocode for choosing the right framework

```
START

Understand the problem

IF data is tabular
    IF task is classification, regression, or clustering
        Use Pandas for cleaning
        Use NumPy for numerical operations
        Use Scikit-learn for model training
    ENDIF

ELSE IF data is image, audio, or complex sequence
    Use PyTorch or TensorFlow
    IF beginner-friendly high-level API is preferred
        Use TensorFlow/Keras
    ELSE
        Use PyTorch
    ENDIF

ELSE IF data is text and task needs NLP or GenAI
    IF pretrained model can solve the problem
        Use Hugging Face
    ELSE IF custom deep learning model is required
        Use PyTorch or TensorFlow
    ENDIF

ENDIF

Evaluate model

IF model performance is good
    Package model
    Deploy using API
    Monitor in production
ELSE
    Improve data, features, or model
ENDIF

END
```

17. Easy framework selection examples

Example 1: Predict house price

Data type: table
Task: regression
Use: Pandas + NumPy + Scikit-learn

Example 2: Classify customer review sentiment

Data type: text
Task: NLP classification
Use: Hugging Face

Example 3: Detect disease from X-ray image

Data type: image
Task: deep learning classification
Use: PyTorch or TensorFlow

Example 4: Group customers by spending behavior

Data type: table
Task: clustering
Use: Pandas + Scikit-learn

Example 5: Build internal document chatbot

Data type: documents/text
Task: RAG/GenAI
Use: Hugging Face + embeddings + vector DB + LLM

18. Common mistakes

Mistake 1: Starting with deep learning for every problem

Not every AI problem needs PyTorch or TensorFlow.

For tabular business data, Scikit-learn is often a better first choice.

Wrong: Use neural network for simple churn prediction immediately
Better: Start with Logistic Regression, Random Forest, or XGBoost-style model

Mistake 2: Ignoring data cleaning

Many beginners focus only on model training.

But in real projects:

Bad data -> Bad model
Clean data -> Better model

Pandas is very important because real enterprise data is messy.

Mistake 3: Confusing NumPy and Pandas

Simple memory:

```
NumPy = numerical arrays  
Pandas = tables and data analysis
```

Use Pandas when data has rows and columns. Use NumPy when you need fast numerical/matrix operations.

Mistake 4: Using Hugging Face without understanding tokens

Hugging Face models work with tokens, not raw text directly.

Text must be tokenized first.

```
Raw text -> Tokens -> Model -> Output
```

Mistake 5: Thinking Scikit-learn is outdated

Scikit-learn is still very useful in enterprise AI.

Many production ML systems still use traditional ML because it is:

- Faster
 - Cheaper
 - Easier to explain
 - Easier to deploy
 - Good for tabular data
-

Mistake 6: Not building a baseline model

Before trying complex models, build a simple baseline.

Example:

```
Step 1: Logistic Regression baseline  
Step 2: Random Forest  
Step 3: Advanced model only if needed
```

Mistake 7: Not thinking about production

In enterprise AI, model training is only one part.

You also need:

```
Data pipeline  
Model tracking  
Model versioning
```

API deployment
Monitoring
Security
Cost control
Governance

19. Interview-friendly explanation

For IBM AI/GenAI roles, you can explain the ecosystem like this:

In AI projects, I use Python as the main language because it connects data processing, ML, deep learning, and GenAI workflows.

For data preparation, I use Pandas and NumPy. For traditional ML problems on structured data, I use Scikit-learn. For deep learning or custom neural networks, I use PyTorch or TensorFlow/Keras. For GenAI and NLP use cases, especially when using pretrained transformer models, I use Hugging Face.

In enterprise projects, these tools fit into a larger lifecycle: data ingestion, preprocessing, model training, evaluation, deployment through APIs, monitoring, and continuous improvement.

20. One simple memory map

NumPy	-> Numbers
Pandas	-> Tables
Scikit-learn	-> Traditional ML
PyTorch	-> Flexible deep learning
TensorFlow	-> Production deep learning
Keras	-> Easy TensorFlow
Hugging Face	-> Pretrained LLM/NLP models

This is the easiest way to remember the Python AI framework ecosystem.